Implantación de sistemas seguros de desplegado de software.

Caso práctico

Julián ha empezado un nuevo trabajo en el departamento de DevOps de una empresa de Ciberseguridad. Dicho departamento trabajará creando aplicaciones y desarrollos junto a los analistas de seguridad del SOC (Centro de Operaciones de Seguridad). El modo de trabajo será de entregas continuas en vez de esperar a que el proyecto esté finalizado. Al estar junto al consumidor de estas aplicaciones, el feedback será continuo entre creador y consumidor y podrán pulirse muchas situaciones antes de llegar a producción, siendo un beneficio para los desarrolladores y los clientes o consumidores.



mr-panda para Pixabay. *devops* (Uso gratuito Licencia Pixabay)

El ciclo de desarrollo de software ha cambiado mucho durante los últimos años, nuevos modelos como DevOps con entregas y alineamientos continuos con las necesidades del cliente han cambiado la metodología e incluso la cultura en el desarrollo. El modelo ha mutado hacia una comunicación continuación el cliente, basado en entregas cortas y un continuo alineamiento con las necesidades reales. Esto ha provocado que el resultado se acerque más a las expectativas del cliente y se pueda cubrir cualquier desviación con mas agilidad.

Por otra parte tenemos que garantizar que cuando implantemos nuestro software sea de la manera más segura y colaborativa posible, garantizando que todos los miembros del equipo pueden escribir su código y que tenemos disponible un control de versiones, un entornos seguro de desarrollo, etc

GOBIERNO DE ESPAÑA
MINISTERIO DE EDUCACIÓN, FORMACIÓN PROFESIONAL Y DEPORTES

Ministerio de Educación, Formación Profesional y Deportes. (Dominio público)

Materiales formativos de FP Online propiedad del Ministerio de Educación, Formación Profesional y Deportes.

Aviso Legal

1.- Puesta Segura en producción y DevOps.

Caso práctico



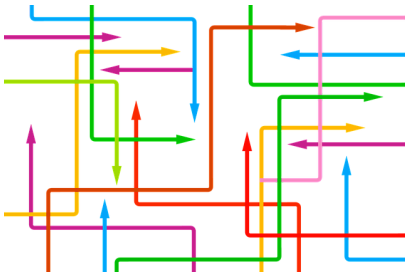
StockSnap. *Portatil, programación código* (Licencia de Pixabay)

Julián acaba de entrar en el departamento DevOps y necesita entender esta metodología. Entiende que la puesta segura en producción del código que producen los desarrolladores ha evolucionado mucho durante los últimos años, ahora los desarrollos son en equipo (es decir, todos los desarrolladores conocen en tiempo real el desarrollo exacto que está haciendo el compañero) y muy cercano al cliente. Esta metodología engloba un conjunto de prácticas que agrupan el desarrollo de software (**Dev**) y las operaciones de TI (**Ops**).

Además, para aprovechar al máximo la agilidad y la capacidad de respuesta de los enfoques de DevOps, la seguridad de la TI también debe desempeñar un papel integrado en el ciclo de vida completo de sus aplicaciones. Surge entonces **DevSecOps** (Dev+ Sec + Ops). Esta práctica implica pensar desde el principio en la seguridad de las aplicaciones y de la infraestructura. También implica automatizar algunos controles de seguridad para impedir que se ralentice el flujo de trabajo de DevOps.

DevOps no es más que una **metodología** basada en la creación de software de forma continua y de mayor calidad mediante versiones más frecuentes. Esto se traduce de forma directa en claras ventajas:

- Las versiones más frecuentes hacen que se pueda alinear la expectativa del cliente de forma más sencilla.
- Cualquier cambio en el proyecto se puede corregir con más facilidad.
- El software tiene más calidad al estar enfocado en objetivos más concretos.



geralt para Pixabay. *flechas-dirección* (Uso gratuito Licencia Pixabay)

El enfoque DevOps abarca varias fases clave a lo largo del ciclo de vida del desarrollo de software, siendo esencial la automatización en todas estas fases para lograr una entrega continua y eficiente. Para cada una de las fases se utilizan multitud de herramientas. A continuación, se muestra un ejemplo de fases y de herramientas:

Fases típicas

Unas fases típicas (puede variar según las necesidades específicas del proyecto y las preferencias del equipo) son:

- **Planificación** (**Plan** en inglés): es donde se establecen los sprints (término que se verá en el siguiente apartado sobre Agile) y se asignan tareas a los desarrolladores.
- **Codificación** (**Code** en inglés): es donde se desarrolla el código y se hacen pruebas unitarias.
- **Construcción** (**Build** en inglés): en la que se utilizan herramientas de automatización para crear los artefactos de lanzamiento. Los artefactos son una copia aislada del código compilado o "configurado" de otro modo que puede desplegarse en los entornos necesarios para pruebas o producción.
- **Pruebas** (**Test** en inglés): la nueva versión del software se somete a una serie de pruebas para comprobar si está lista para la producción.
- **Lanzamient**o (**Release** en inglés): el objetivo principal es garantizar que la infraestructura y el entorno estén preparados para lanzar con éxito el software actualizado.
- **Despliegue** (**Deploy** en inglés): en esta fase el equipo de operaciones se coordinará para copiar los artefactos de lanzamiento que se han creado y probado en los servidores de producción.
- **Funcionamiento** (**Operate**, en inglés): en esta fase el equipo de operaciones debe garantizar que los componentes de la infraestructura permanezcan en línea y crezcan con la demanda según sea necesario.
- **Monitorización** (**Monitor** en inglés): la recopilación y análisis de los datos analíticos generados desde el propio software (conexiones activas, registro de accesos, errores, ...), junto con los comentarios de los usuarios y la revisión de los procesos permite garantizar el buen funcionamiento del mismo y realizar mejoras en el mismo.

Herramientas típicas

En lugar de una sola herramienta de DevOps existen conjuntos ("toolchains" en inglés) de múltiples herramientas. Como ejemplo (hay que tener en cuenta que hay muchas otras opciones disponibles y la elección de herramientas puede variar según las necesidades específicas del proyecto y las preferencias del equipo) de herramientas típicas:

Control de versiones	Git
Automatización de la construcción	Jenkins
Pruebas	Selenium
Gestión de la configuración	Terraform
Implementación	Kubernetes
Monitorización	ELK Stack

Una metodología efectiva de DevOps garantiza ciclos de desarrollo rápidos y frecuentes (a veces de semanas o días), pero debe integrar la seguridad en todas la fases para que además sea segura. Surge así **DevSecOps**: Desarrollo (Dev) + Seguridad (Sec) +

Operaciones (Ops). Este enfoque no solo requiere incorporar herramientas nuevas, sino también adoptar un enfoque empresarial distinto. Los equipos de DevOps deben tener esto en cuenta al automatizar la seguridad para proteger el entorno y los datos por completo, así como el proceso de integración y distribución continuas. Se busca fomentar una cultura colaborativa donde los equipos trabajan juntos para identificar y abordar proactivamente vulnerabilidades, permitiendo la entrega rápida y segura de software en un entorno dinámico y en constante evolución.

Debes conocer

Para ampliar más sobre los conceptos de DevOps y DevSecOps te animamos a visitar enlace de Red Hat

Para saber más

Durante los últimos años ha surgido junto a la metodología de DevOps nuevas maneras de gestionar proyectos y entregas con clientes basadas en metodologías ágiles (*agile* en inglés). Si bien ambas tienen algunos principios similares y pueden ser complementarias es conveniente saber las diferencias y puntos de convergencia. Te dejamos este enlace si quieres saber más sobre el tema.

1.1.- Desarrollo Ágil.



Agile Manifesto (Captura de Pantalla)

Como comentábamos la manera de desarrollar software ha cambiado, necesitamos adaptarnos de forma sencilla ante los cambios del cliente y para poder hacerlo surgen las metodologías ágiles en el desarrollo. Estas metodologías se basan en una serie de principios básicos:

1.- Satisfacción del cliente mediante la entrega temprana y continua de software de calidad.
2.- Gran aceptación y respuesta al cambio de los requisitos definidos durante el proyecto.
3.- Entrega frecuente de software funcional, periodos 2-6 semanas.
4.- Trabajo conjunto entre el cliente y desarrolladores.
5.- Creación de equipos motivados y respaldados.
6.- Comunicación cara a cara desarrolladores-cliente.
7.- Equipos auto-organizados.
8.- Reflexión continua de formas para mejorar la efectividad y realizar los ajustes necesarios para corregir cualquier desviación.

De entre todas ellas surge **Scrum,** una metodología ágil que engloba una serie de principios que ayudan a los equipos a desarrollar productos en periodos cortos (llamados sprints), permitiendo obtener de forma rápida *feedback*, adaptaciones y una mejora continua.

Dentro de los Sprints también se hacen ciclos de verificación o bucles, y simulación de fallos. Estos bucles ayudan a verificar la calidad del software y tomar las medidas oportunas sino fuese así, ayudando a mejorar la calidad del software. Además, estos bucles retroalimentan el proceso, mejorando la calidad del producto final.

Debes conocer

Existe un manifiesto que marca los principios fundamentales de la metodología ágil en el desarrollo de software. No especifica el cómo, pero si define los principios fundamentales. Puedes consultarlo aquí .

Para saber más

Unos de los bucles o ciclos más famosos es el ciclo Deming, que llevó a Toyota a convertirse en uno de los gigantes de la industria del automóvil y a Japón ser un referente de la industria. Se utiliza también dentro de la metodología Scrum como un proceso de mejora continua. Puedes consultar más info aquí .

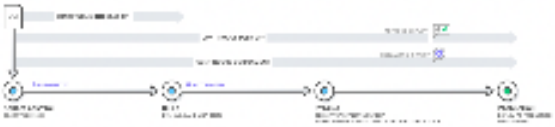
2.- Control de versiones.

Caso práctico

Julián entiende que uno de los principales cambios que surgen con la adopción de la metodología de DevOps es el continuo desarrollo de software mediante lanzamientos o *releases* continuos. Si a todo este proceso de nuevas versiones y lanzamientos unimos a varios miembros del equipo editando, codificando y generando cambios en las distintas piezas del software, hace que sea fundamental poder disponer de un modelo automático de control de versiones y cambios.



StockSnap, *Portatil, programación código* (Licencia de Pixabay)



Amazon AWS (Captura de pantalla)

Los equipos de desarrollo trabajan con sistemas de control de versiones (o VCS, por sus siglas en inglés), que son herramientas software que gestionan, administran y controlan (creación, adición, eliminación, modificación del código) sobre un **repositorio** común.

Este repositorio está disponible para todos los desarrolladores y ofrece una manera colaborativa de trabajar donde todos los programadores pueden tener acceso al código y conocer qué cambios se han hecho, quién y cuándo los ha hecho y el estado actual de la versión. También proporciona una manera sencilla de tener un **backup del código y recuperarnos** de algún cambio erróneo o problema. Este repositorio es el resultado visible de lo que se conoce como **Integración continua.**

La integración continua es una práctica de desarrollo de software mediante la cual los desarrolladores combinan los cambios en el código en un repositorio central de forma periódica, tras lo cual se ejecutan versiones y pruebas automáticas.

Las principales ventajas del **Control de Versiones** son**:**

- Revertir y corregir cambios de manera sencilla.
- Copia de seguridad o backup del código.
- Resolución de conflictos cuando uno o más miembros modifiquen el mismo fichero a la vez.

Algunos ejemplos de herramientas populares para el control de versiones son:

- **Git**: Permite a los equipos de desarrollo llevar un registro de los cambios en el código fuente a lo largo del tiempo, trabajar en ramas de manera eficiente y colaborar en proyectos de manera efectiva. GitHub y GitLab son plataformas populares que utilizan Git para gestionar repositorios de código.
- **Subversion (SVN)**: Aunque ha perdido algo de popularidad frente a sistemas distribuidos como Git, SVN sigue siendo una herramienta de control de versiones centralizada muy utilizada. Permite a los desarrolladores mantener un historial de versiones y realizar seguimiento de cambios en archivos y directorios.

Debes conocer

La integración continua está estrechamente relacionada con el control de versiones; los desarrolladores envían los cambios sobre el software de forma periódica al repositorio compartido con un sistema de control de versiones. La combinación de ambas soluciones proporciona la integración continua proporcionada por un repositorio común que se mantiene actualizado y, además, el control de versión asegura tener todos los cambios controlados.

Uno de los sistemas de control de versiones más famosos y que se ha convertido en el estándar es **Git**, es un sistema de control de versiones distribuido usado por los desarrolladores de software. Tienes más información en este enlace de wikipedia.

En Git hay tres etapas primarias (condiciones) en las cuales un archivo puede estar: estado **modificado**, estado **preparado**, o estado **confirmado**.

Mediante Git podremos **clonar un repositorio** o incluso si queremos probar el software podremos **construirlo (build**) de manera automática usando el mismo código que han usado los desarrolladores para crearlo.

No hay que confundir **Git** con **Github**, esta última es una plataforma basada en la web donde los usuarios pueden alojar repositorios Git. Facilita compartir y colaborar fácilmente en proyectos con cualquier persona en cualquier momento.

Autoevaluación

Identifica si las siguientes frases son verdaderas o falsas

DevOps es un nuevo estándar de programación.

○ Verdadero ○ Falso

Falso

DevOps es una metodología de desarrollo que aporta numerosos beneficios.

Hoy en día los desarrolladores funcionan de forma aislada.

○ Verdadero ○ Falso

Falso

Las desarrolladores suelen trabajar en grupo en la creación de software.

DevOps ha cambiado la manera en que se entrega el software al cliente.

○ Verdadero ○ Falso

Verdadero

La metodología de DevOps mejora la calidad del software mediante entregas más rápidas al cliente.

Para que todos los desarrolladores trabajen de forma colaborativa y eficiente surgen los repositorios.

○ Verdadero ○ Falso

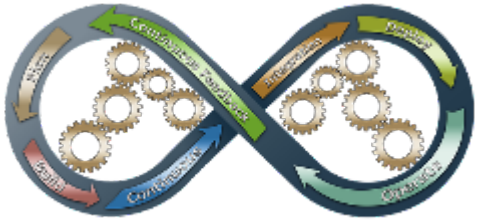
Verdadero

Los repositorios son la manera estándar de almacenar el código y trabajar de forma colaborativa.

3.- Sistemas de automatización de construcción

Caso práctico

Julián se ha dado cuenta que la automatización es una de las principales claves en DevOps, no sólo por la reducción de tiempos, sino que también minimizan el riesgo de errores. En el caso de la entrega y despliegue continuo (CI/CD) ha encontrado varias herramientas que permiten automatizar esta tarea y va a realizar unas pruebas con cada una para ver cuál es la mejor que se adapta a los proyectos con los que trabaja.



mr-panda para Pxabay. *DevOps* (Uso Gratuito Licencia Pixabay)

La entrega y despliegue continuo (CI/CD) son prácticas fundamentales en el ámbito de DevOps que buscan optimizar y acelerar el ciclo de vida del desarrollo de software. **La entrega continua** implica la automatización de los procesos de construcción, prueba y empaquetado, garantizando que el código probado y funcional esté siempre disponible para ser desplegado. Por otro lado, el **despliegue continu**o lleva esta automatización un paso más allá al liberar automáticamente las nuevas versiones del software en entornos de producción después de pasar las pruebas pertinentes.

Existen diversas herramientas para automatizar las tareas CI/CD, como la compilación, prueba y empaquetado de aplicaciones, proporcionando flujos de trabajo consistentes y repetibles. Al eliminar la intervención manual en estos procesos, los sistemas de automatización de la construcción contribuyen a la reducción de errores, la aceleración del tiempo de entrega y la garantía de la coherencia en todo el ciclo de vida del desarrollo de software. Estas soluciones permiten a los equipos centrarse en la innovación y el desarrollo, mientras aseguran la entrega de software de alta calidad de manera eficiente

Veamos algunas de las más utilizadas:

Jenkins



The Jenkins project. *Logo Jenkins* (CC BY-SA)

Jenkins es una plataforma de automatización de código abierto diseñada para facilitar la integración continua y la implementación continua (CI/CD). Funciona como un servidor de automatización que permite a los equipos definir, programar y ejecutar flujos de trabajo automatizados, desde la compilación y prueba hasta la implementación en entornos de producción. Utilizando una arquitectura extensible y una amplia gama de complementos, Jenkins se integra fácilmente con diversas herramientas y servicios, permitiendo la orquestación de procesos complejos. Los conceptos clave de Jenkins incluyen la creación de trabajos, la gestión de pipelines, y la posibilidad de monitorear y visualizar el estado de los procesos de construcción y despliegue.

Para ampliar visita su página web en este enlace enlace

Circle Ci



FelicianoTech. *Logo Circle CI* (CC BY-SA)

Circle CI es una plataforma de integración continua y entrega continua (CI/CD) basada en la nube. Permite a los equipos de desarrollo automatizar el proceso de construcción, prueba y despliegue de aplicaciones en entornos diversos. CircleCI utiliza pipelines configurables para definir flujos de trabajo personalizados, lo que facilita la orquestación de tareas complejas. Con soporte para diversos lenguajes de programación y frameworks, CircleCI se integra fácilmente con repositorios de código alojados en plataformas como GitHub y Bitbucket. Además, su enfoque en la escalabilidad y la velocidad lo convierte en una opción popular para proyectos de diferentes tamaños, permitiendo a los equipos acelerar el ciclo de vida del desarrollo y mantener la calidad del software a lo largo del tiempo.

Para ampliar visita su página web en este enlace enlace

Github Actions



Github. *Github Actions Logo* (Github)

GitHub Actions es una plataforma de automatización integrada directamente en el entorno de GitHub, diseñada para facilitar la automatización de flujos de trabajo en el desarrollo de software. Permite a los desarrolladores definir, personalizar y compartir flujos de trabajo que abarcan desde la integración continua hasta la implementación continua.

Para ampliar visita su página web en este enlace enlace

Debes conocer

En el contexto de CI/CD se utiliza habitualmente el termino "pipeline". Es una representación visual y automatizada del flujo de trabajo que sigue el código desde su desarrollo hasta su implementación. Se compone de una serie de pasos interconectados que definen las fases del ciclo de vida del software, como la compilación, las pruebas y el despliegue.

OWASP tiene un proyecto para securizar los pipeline en este enlace .

4.- Integración continua y automatización de pruebas.

Caso práctico

Al utilizar las herramientas de automatización de la construcción CI/CD **Julián** se ha dado cuenta que las pruebas son la clave fundamental para la seguridad, pero estas deben ser automatizadas para no ralentizar los procesos y evitar errores. Ha decidido probar varias herramientas de automatización de pruebas, tanto funcionales como de seguridad, para comprender mejor su funcionamiento, sus ventajas y su problemática.



StockSnap. *Portatil, programación código* (Licencia de Pixabay)

La automatización de pruebas en CI/CD juega un papel crucial en la mejora de la calidad del software y la eficiencia del proceso de desarrollo. Esta práctica implica la creación y ejecución automatizada de pruebas a lo largo del ciclo de vida del software, desde la integración continua hasta la entrega continua. Al incorporar pruebas automatizadas en el pipeline de CI/CD, los equipos pueden identificar rápidamente posibles problemas y garantizar que las nuevas actualizaciones no introduzcan regresiones.

Las pruebas automatizadas pueden abarcar desde pruebas unitarias hasta pruebas de integración y pruebas de extremo a extremo, proporcionando una cobertura integral. La automatización de pruebas no solo acelera el proceso de desarrollo, sino que también contribuye a la confiabilidad y estabilidad del software al garantizar que los errores se identifiquen y aborden de manera proactiva en cada etapa del ciclo de vida del desarrollo.

Veamos algunos ejemplos típicos de herramientas para automatizar pruebas en función del tipo:

Pruebas unitarias

Ya las vimos en la primera unidad en detalle. Estas pruebas están diseñadas para validar que cada componente del software realiza sus tareas de manera aislada y produce los resultados esperados. Estas pruebas no sólo se deben realizar durante el desarrollo, también deben realizarse en el CI/CD para asegurarse de que el software es correcto y seguro.

Ejemplos de herramientas para pruebas automatizadas unitarias: **PHPUnit, JUnit, NUnit, pytest, Jasmine**

Pruebas de Integración

Recuerda que estas pruebas verifican que las unidades de código, previamente probadas de manera individual en las pruebas unitarias, funcionen correctamente cuando se combinan. El objetivo principal de las pruebas de integración es identificar posibles conflictos en la interfaz y asegurar que los distintos elementos del software trabajen de manera conjunta sin problemas.

Ejemplos de herramientas para pruebas automatizadas de integración: **TestNG, Mocha, Protractor**

Pruebas de Aceptación

Recuerda que estas pruebas se centran en evaluar el comportamiento global de la aplicación desde la perspectiva del usuario final, verificando que todas las funciones y características se desempeñen según lo esperado. Al integrar pruebas de aceptación en un *pipeline* de CI/CD, los equipos pueden asegurarse de que las nuevas implementaciones no solo sean técnicamente sólidas, sino también que cumplan con las expectativas del usuario. Piensa en este *testing* como un robot que usa nuestra aplicación, simulando acciones y peticiones realizadas desde un navegador.

Ejemplos de herramientas para pruebas automatizadas de acepetación: **Selenium WebDriver, Cucumber, Cypress**

Pruebas de Seguridad

Estas pruebas buscan identificar y mitigar posibles vulnerabilidades y riesgos de seguridad en el software. Al incorporar pruebas de seguridad en el *pipeline* de CI/CD, los equipos pueden automatizar la evaluación de posibles amenazas, como vulnerabilidades conocidas, configuraciones inseguras o prácticas de codificación deficientes.

Ejemplos de herramientas para pruebas automatizadas de seguridad: **OWASP ZAP, Burp Suite, Nessus, Acunetix, SonarQube, Docker scan**

La elección de una herramienta específica puede depender de los requisitos del proyecto, las preferencias del equipo y otros factores específicos.

5.- Virtualización y contenedores.

Caso práctico

Uno de los problemas que se presentan durante el desarrollo y despliegue de aplicaciones es la heterogeneidad entre los sistemas de desarrollo, no ya entre los propios desarrolladores, también entre los sistemas de desarrollo, entornos de pruebas, preproducción y producción. Los sistemas donde se crea y se prueba el software tienen librerías y sistemas operativos diferentes. Esto provoca que la aplicación o software pudiera funcionar bien con unas determinadas versiones de librerías o sistemas operativos, pero no en otros, o ejecutarse bien con la versión de un lenguaje, pero no con otra. Para asegurar la calidad de desarrollo tenemos que asegurar que todo el equipo de desarrollo usa las mismas versiones de todas las aplicaciones y librerías necesarios.

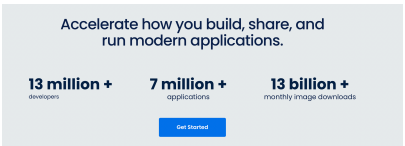


Hidro. *VirtualBox2* (CC BY-SA)

Esto es más complicado de lo que parece, porque hay desarrolladores que prefieren una distribución concreta, o incluso sistemas privativos. Incluso los sistemas de pruebas, preproducción y producción suelen ser distintos. Los sistemas de producción suelen ser más potentes y los básicos se dejan para pruebas y preproducción. Otro problema derivado es que los equipos desarrollo a veces están involucrados en varios proyectos a la vez y cada uno de estos proyectos requiera versiones distintas de librerías, complicándolo aún más. De hecho, podría pasar que cambios que se hagan en una librería que afecta a un proyecto acabe afectando sin querer a otro proyecto, suponiendo un gran problema.

Para solucionar estos problemas es importante que los desarrolladores creen el código en entornos controlados y fácilmente replicables. **Julián** siempre ha trabajado con **máquinas virtuales**, pero en el nuevo proyecto se tiene que enfrentar al despliegue de una aplicación utilizando **contenedores**.

Las máquinas virtuales (VMs de sus siglas en inglés) y los contenedores son dos tecnologías de virtualización que comparten el objetivo común de optimizar la eficiencia y flexibilidad en el despliegue de aplicaciones, pero difieren en sus enfoques y características fundamentales.



Docker (Captura de pantalla)

Las máquinas virtuales son entornos completamente aislados que emulan sistemas operativos completos dentro de un hipervisor. Cada VM incluye un sistema operativo propio, así como recursos de sistema dedicados. Aunque proporcionan un alto nivel de aislamiento y permiten ejecutar aplicaciones en entornos

distintos, el enfoque de las VMs a menudo implica un mayor consumo de recursos y tiempos de inicio más lentos debido a la necesidad de cargar un sistema operativo completo.

Por otro lado, los contenedores son entornos ligeros que comparten el mismo núcleo (kernel) del sistema operativo subyacente y, por lo tanto, son más eficientes en términos de recursos y tiempos de inicio. Los contenedores encapsulan solo las bibliotecas y dependencias necesarias para ejecutar una aplicación específica, lo que permite una rápida implementación y escalabilidad. Se pueden exportar y usar en cuestión de segundos y hace que todo el equipo de desarrollo trabaje con las mismas herramientas, y en el caso de que estén en varios proyectos usar contenedores distintos.

La elección entre máquinas virtuales y contenedores depende de los requisitos específicos del proyecto. Las VMs son ideales para situaciones que requieren un aislamiento más estricto, como la ejecución de aplicaciones con sistemas operativos diferentes, mientras que los contenedores son más adecuados para implementaciones ágiles y escalables donde la eficiencia de recursos es primordial. Ejemplos comunes de plataformas de máquinas virtuales incluyen **VMware, KVM y VirtualBox**, mientras que **Docker** es una herramienta popular para la gestión de contenedores.



dotCloud, Inc., *Docker Logo* (Apache License 2.0)

Veamos un poco más en detalle **Docker**:

- Se basa en el uso de imágenes. Una imagen es esencialmente una representación ligera y portátil de un sistema de archivos que incluye la aplicación y sus dependencias.
- Un contenedor es una **imagen** ejecutándose en un sistema. Por ejemplo, una imagen de Debian con Apache Tomcat + Mysql. Dicha imagen se puede lanzar en un sistema Ubuntu creando un contenedor. Dicho contenedor podrá tener su propia dirección IP y compartirá los recursos *hardware* con el resto de aplicaciones que se están ejecutando en el sistema (al lanzar el contenedor también se pueden fijar límites de uso de memoria, CPU, ...).
- Docker tiene un registro central con todo tipo de imágenes (**Docker Hub**). Además, cualquier imagen o incluso un contenedor ejecutándose puede convertirse en una nueva imagen.
- Docker ofrece herramientas de gestión de cluster con **Swarm** y **Compose**.
- La **arquitectura de docke**r está compuesta de:
  - El **cliente de Docker** (Docker Client en inglés) es la principal interfaz de usuario para Docker. Acepta comandos del usuario y se comunica con el demonio Docker.
  - El **demonio Docker** (Docker Engine en inglés) corre en una maquina anfitriona (host). El usuario no interactúa directamente con el demonio, en su lugar lo hace a través del
    cliente Docker.
  - El demonio Docker levanta los contenedores haciendo uso de las imágenes, que pueden estar en local o en el **Docker Registry** (repositorio de imágenes).

Debes conocer

Puedes conocer más sobre la tecnología Docker y sus numerosos beneficios en este enlace .

6.- Gestión automatizada de configuración de sistemas

Caso práctico



StockSnap. *Portatil, programación código* (Licencia de Pixabay)

En organizaciones con gran cantidad de máquinas (físicas o virtuales) uno de los mayores desafíos es la tarea de automatizar los procesos para preparar la infraestructura, gestionar la configuración, implementar las aplicaciones y organizar los sistemas, mejorar la seguridad y el cumplimiento, ejecutar parches en los sistemas y compartir la automatización en toda la empresa, entre otros procedimientos de TI.

En el proyecto en el que está trabajando **Julián** hay demasiadas máquinas que gestionar, por lo que se ha puesto a investigar herramientas para gestionar la configuración. Estas herramientas se basan en archivos de configuración que se deben versionar y almacenar en repositorios de control de versiones como Git.

Las herramientas de gestión automatizada de configuración de sistemas aseguran la consistencia y la integridad de la configuración a lo largo del tiempo, facilitan la automatización de tareas repetitivas y simplifican la implementación y actualización de software.

Estas plataformas permiten a los equipos definir, gestionar y aplicar configuraciones de manera centralizada, lo que resulta esencial para mantener entornos de infraestructura complejos de manera eficiente y segura. Al utilizar estas herramientas, los equipos pueden garantizar que los sistemas se configuren de manera coherente, reduciendo el riesgo de errores y mejorando la seguridad y la estabilidad del entorno operativo.

Veamos algunas de estas herramientas:

Ansible

Ansible es una plataforma de automatización de código abierto que se centra en la gestión de configuración, la implementación de aplicaciones y la automatización de tareas. Desarrollado en Python, Ansible utiliza un enfoque basado en la infraestructura como código (IaC), lo que significa que las configuraciones y tareas se definen en archivos de configuración. A través de un modelo de ejecución basado en **SSH**, Ansible permite a los usuarios especificar cómo deberían estar configurados los



Ansible.com.
Ansible logo
(Dominio público)

sistemas y servicios, y luego aplica esas configuraciones de manera consistente a través de todos los nodos del sistema. Su diseño sin agente (también admite con agente pero no es la opción habitual) y su arquitectura modular lo hacen altamente escalable y fácil de implementar.

Para ampliar más visita su página web en este enlace

Puppet



™/®Puppet Labs.
Puppet Logo (Dominio público)

Puppet es una herramienta de gestión de configuración de código abierto diseñada para automatizar y gestionar la configuración de sistemas informáticos. Utiliza un enfoque declarativo, donde los administradores definen el estado deseado de los sistemas, y Puppet se encarga de llevarlos a ese estado de manera coherente. Escrito en **Ruby**, Puppet ofrece un lenguaje específico del dominio (DSL) para describir configuraciones y políticas. Utilizando un modelo cliente-servidor, los nodos gestionados por Puppet se conectan a un servidor maestro que distribuye y aplica las configuraciones definidas. Puppet es ampliamente utilizado para gestionar la infraestructura, desde la configuración de servidores hasta la instalación de software y la gestión de recursos de red.

Para ampliar más visita su página web en este enlace

Chef



Progress Chef.
Progress Chef logo
(Todos los derechos reservados)

Chef es una herramienta de automatización de TI que se enfoca en la gestión de configuración y la automatización de la infraestructura. Utilizando un enfoque basado en código, Chef permite a los administradores describir la configuración deseada de los sistemas en forma de "recetas" y "roles". Estas recetas definen cómo deben configurarse y qué software debe estar instalado en cada nodo del sistema. Chef opera en un modelo cliente-servidor, donde los nodos gestionados se conectan a un servidor Chef para obtener y aplicar las configuraciones definidas. Chef destaca por su flexibilidad y su capacidad para gestionar tanto entornos pequeños como grandes a escala.

Para ampliar más visita su página web en este enlace

Para saber más

Puedes visitar este enlace para ver una comparativa entre Ansible, Puppet y Chef

7. Herramientas de simulación de fallos

Caso práctico

La simulación de fallos es una práctica esencial en el ámbito de la ingeniería de software y la gestión de sistemas que consiste en reproducir condiciones adversas de manera controlada para evaluar la resistencia y la capacidad de recuperación de un sistema. Esta técnica permite probar la robustez de una aplicación o infraestructura ante situaciones inesperadas, como fallos de hardware, caídas de red o pérdida de datos. Al simular fallos de manera deliberada, los equipos pueden identificar vulnerabilidades potenciales, mejorar la resiliencia y desarrollar estrategias efectivas de recuperación.



StockSnap. *Portatil, programación código* (Licencia de Pixabay)

En el nuevo proyecto le han encargado a **Julián** investigar sobre herramientas de simulación de fallos

Herramientas y prácticas como "Chaos Engineering" se han vuelto populares para implementar de manera controlada y gradual la simulación de fallos, permitiendo a las organizaciones fortalecer sus sistemas y servicios en condiciones adversas simuladas antes de enfrentar eventos reales no planificados. Este enfoque proactivo contribuye a la mejora continua de la fiabilidad y la disponibilidad de los sistemas críticos.

Un ejemplo paradigmático de esta metodología es la implementación de pruebas de caos, como las conducidas por herramientas como **Chaos Monkey** en entornos basados en microservicios. Estas pruebas introducen deliberadamente fallos en nodos o servicios en producción para evaluar la capacidad del sistema para resistir y recuperarse de eventos inesperados. La simulación puede incluir, por ejemplo, la desconexión de nodos, la introducción de retrasos en la red o la generación de errores en la capa de aplicación. Al exponer un sistema a escenarios de fallos controlados, los equipos pueden descubrir debilidades potenciales, optimizar la recuperación automática y, en última instancia, mejorar la resiliencia general del sistema frente a eventos disruptivos del mundo real



Netflix. *Chaos Monkey logo by Netflix* (Apache License 2.0)

Para saber más

Si quieres ampliar sobre Chaos Monkey te recomiendo visitar el proyecto de Neftlix en este enlace

8.- Orquestación de contenedores.

Caso práctico

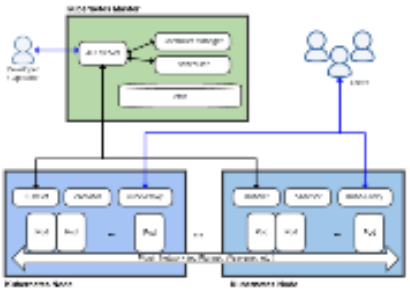
Un orquestador asume la responsabilidad de coordinar y automatizar el despliegue, configuración y gestión de servicios y aplicaciones en una arquitectura distribuida. Estos orquestadores facilitan la escalabilidad, la recuperación de fallos y la gestión eficiente de recursos al proporcionar una visión centralizada y coherente del estado de la infraestructura. Al permitir la automatización de tareas complejas y secuencias de trabajo, los orquestadores juegan un papel clave en la optimización de la eficiencia operativa y en la implementación efectiva de sistemas distribuidos en entornos empresariales modernos.

En el proyecto actual **Julián** tiene que desplegar una aplicación web en un orquestador **Kubernetes** en un entorno cloud.

kubernetes

Unknown author. *Logo Kubernetes* (Dominio público)

Un **orquestador** es una herramienta o plataforma que se encarga de coordinar y gestionar la ejecución de procesos o servicios de manera eficiente y automatizada en un entorno computacional. Los orquestadores son esenciales en entornos complejos y distribuidos, donde múltiples componentes deben trabajar juntos de manera coordinada. Hay varios tipos de orquestadores, cada uno diseñado para abordar necesidades específicas en diferentes contextos. En la imagen de la derecha puedes ver el esquema de funcionamiento de uno de ellos.



Khtan66. *Kubernetes* (CC BY-SA)

A continuación se presentan algunos tipos comunes:

Orquestador de Infraestructura

Terraform

HashiCorp. *Terraform Logo* (Mozilla Public License Version 2)

Los orquestadores de infraestructura, como **Terraform** y **AWS CloudFormation**, se centran en la provisión y gestión de recursos de infraestructura en la nube, como servidores virtuales, redes, bases de datos y otros servicios.
Trabajan a un nivel más bajo, proporcionando una

capa de abstracción sobre los recursos físicos y permitiendo la definición y gestión de la infraestructura como código (IaC).
Se centran en máquinas virtuales, almacenamiento y redes.

svg-logos. *Logo AWS CloudFormation* (CC0)

Orquestador de Contenedores



Unknown author. *Logo Kubernetes* (Dominio público)

Los orquestadores de contenedores, como **Kubernetes** y **Docker Swarm**, se diseñaron para gestionar el ciclo de vida de aplicaciones *contenerizadas*, desde la implementación y escalado hasta la administración de recursos y la orquestación de servicios.



abhinavtripathy. *Docker Swarm logo* (MIT License)

Operan en un nivel más alto, enfocándose en contenedores y aplicaciones, ofreciendo abstracciones que facilitan la gestión y la orquestación de múltiples instancias de aplicaciones.
Están especializados en gestionar el despliegue y la escalabilidad de contenedores, proporcionando funcionalidades específicas para la orquestación eficiente de aplicaciones *contenerizadas*.

Además, suelen realizar las siguientes tareas:

- Búsqueda de servicios (Service Discovery en inglés) que permite acceder a los distintos contenedores del cluster.
- Balanceo de carga.
- Configuración de la red.
- Escalabilidad.
- Registro (Logging) y Monitoreo.
- Respuesta a fallos.

Ambos tipos de orquestadores son herramientas que desempeñan roles distintos, pero a menudo se utilizan de manera complementaria en entornos de desarrollo y operaciones.